CIS 5520 Project proposal

- Group members: Steven Galban, Huynh Quynh Anh
- Github ids: sgalban, hqanhh
- Project title: Puzzle Solver

1. Overall goal

Consider the family of number puzzles containing games such as Sudoku, Kakuro, and Magic Squares. While they all have different rules, at their core, they can all be modeled as a grid of numbers subject to certain constraints. This project aims to take advantage of these similarities, generalizing these puzzles into a common framework, and then using that framework to create a general puzzle solver. Specifically, the goal of this project is to create a domain-specific language, Puzzle, that can be used to describe a family of similar grid-based number puzzles (including Sudoku, Kakuro, Magic Squares, and more), and then write a small Haskell library with three primary functions:

1. To parse this domain-specific language into a Haskell persistent data structure.
2. To convert puzzles back into Puzzle code.
3. To determine the solution of an unsolved puzzle (or indicate that no such valid solution exists).

This library will be complete with a command-line interface from which users can perform all the functionality described above.

2. Use case

The use cases for this library are fairly straightforward. For example, from the command-line interface, a user can load a Puzzle file via its filepath, which parses the puzzle and loads it into memory. Once the puzzle is loaded, the user can then modify it, or solve it. For example, suppose the user wants to solve the following 4x4 Sudoku puzzle:

			3
	4		
		3	2

To solve this puzzle, the user would first load the following Puzzle code:

```
grid(4, 4) {
    # Each row has unique entries
    repeat(0, 3) {
        constraint unique(1 to 4) in row $0
    }
    # Each column has unique entries
    repeat(0, 3) {
        constraint unique(1 to 4) in col $0
    }
    # Each box has unique entries
    repeat(0, 3) {
        constraint unique(1 to 4) in subgrid((($0 % 2) * 2, ($0 // 2) * 2), 2, 2)
    }
    init cell(0, 4) with 3
    init cell(1, 1) with 4
    init cell(2, 2) with 3
    init cell(2, 3) with 2
}
```

then simply use the solve command. Alternatively, a user could build this puzzle from scratch within the library and convert it into the Puzzle code shown (or functionally equivalent code). Other types of puzzles that can be modeled by Puzzle code, and are therefore compatible with the library, include but are not limited to, Kakuro, Magic Squares, KenKen, Killer Sudoku, and Futoshiki.

3. Project architecture

This project covers multiple separate domains, and lends itself well to being organized over several modules:

1. **Puzzle**:
   At the core of this project is the `Puzzle` model, which represents both an unsolved puzzle, and parsed Puzzle code. That is, it describes the shape of the grid, its initial values, and all the constraints the puzzle is subject to. This module exposes functions that can be used to add, remove, and modify constraints (persistently), as well as a function to validate the structure of a puzzle, i.e. to ensure that every cell of a puzzle either has an initial value, or is subject to at least one constraint. `Puzzle` will be an instance of `Pretty`, its string representation being Puzzle code that could be used to describe the puzzle. This module will also contain a separate method that converts a puzzle into an ASCII representation of the grid.
   Additionally, this module will contain the `PuzzleSolution` type, which is essentially a pair containing a `Puzzle`, and a mapping from cell coordinates to values used to fill the cells. This model represents total and partial solutions to the puzzle.

2. **PuzzleSolver**:
   This module handles solving the puzzle. Its primary purpose is to expose two functions: one that solves a puzzle, and one that validates a full or partial solution.

   The `solve` function will accept a `Puzzle` as input, and output an `Maybe PuzzleSolution`, containing a complete valid solution (or `Nothing`, if no valid solution exists). Note that solving many of the puzzles described by this DSL are NP-Complete problems, so the solver will not be particularly fast for larger puzzles, but the implementation may use various techniques to reduce the runtime as much as feasibly possible.

   The `validate` function takes a `PuzzleSolution` and returns true if and only if the solution does not violate any of the puzzle's constraints. A fully solved puzzle is a `PuzzleSolution` containing a value for every cell.

3. **PuzzleParser**:
   The parser module exposes only one function, `parsePuzzle`, which accepts a filename for a Puzzle file as input and parses its contents into an `Either Puzzle`, where the `Left` case represents parsing errors.

4. **PuzzleCLI**:
   The CLI module ties the other modules in this project together, allowing users to access the above functionality without writing any of their own Haskell code. Specifically, this module will allow users to load puzzles from the Puzzle code files, solve those puzzles, modify them, and print them in various formats.

4. Testing

Most of the testing for the project will be focused on the Puzzle solver, as valid solutions have some nice properties that lend themselves well to QuickCheck. For example:

- If the `solve` function outputs a `Just PuzzleSolution`, the solution should be valid.
- If a puzzle has a valid structure and an empty solution, its solution is valid.
- If any cell from a valid (partial) solution is cleared, the result should still be valid.
- if any blank cell from an invalid (partial) solution is added, the result should still be invalid.

Additionally, the parser module can contain at least one roundtrip QuickCheck property, ensuring that converting a `Puzzle` into Puzzle code and parsing it back results in an equivalent `Puzzle`. Of course, both of these modules will contain plenty of unit tests as well.